

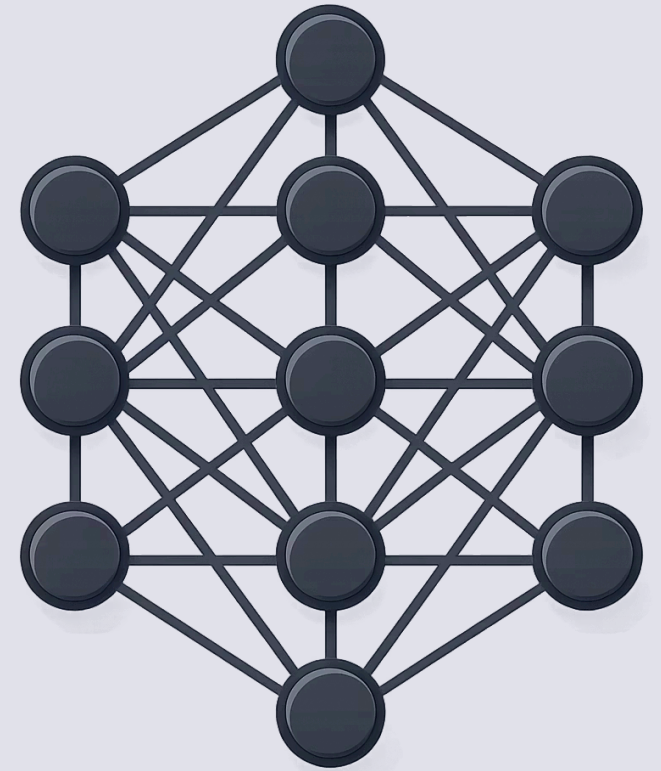
PyTorch — From Basic to Advanced

A complete beginner-friendly guide to building Neural Networks with PyTorch

Dr. Anilkumar V. Brahmane

Computer Engineering Department

Sanjivani College of Engineering





What is PyTorch?

PyTorch is an **open-source deep learning framework** developed by Meta (Facebook AI Research). In simple words: *PyTorch is a Python library that helps us build, train, and use Neural Networks.*



Machine Learning & Deep Learning

Build and train intelligent models



Computer Vision & NLP

Images, text, language models



Generative AI & LLMs

ChatGPT-style models, diffusion models



Research & RL

Reinforcement learning & AI research

Why Do We Need PyTorch?

✗ Without PyTorch

You must manually code:

- Matrix calculations
- Derivatives & gradients
- Weight updates
- GPU computation
- Optimization — repeated thousands of times!

✓ With PyTorch

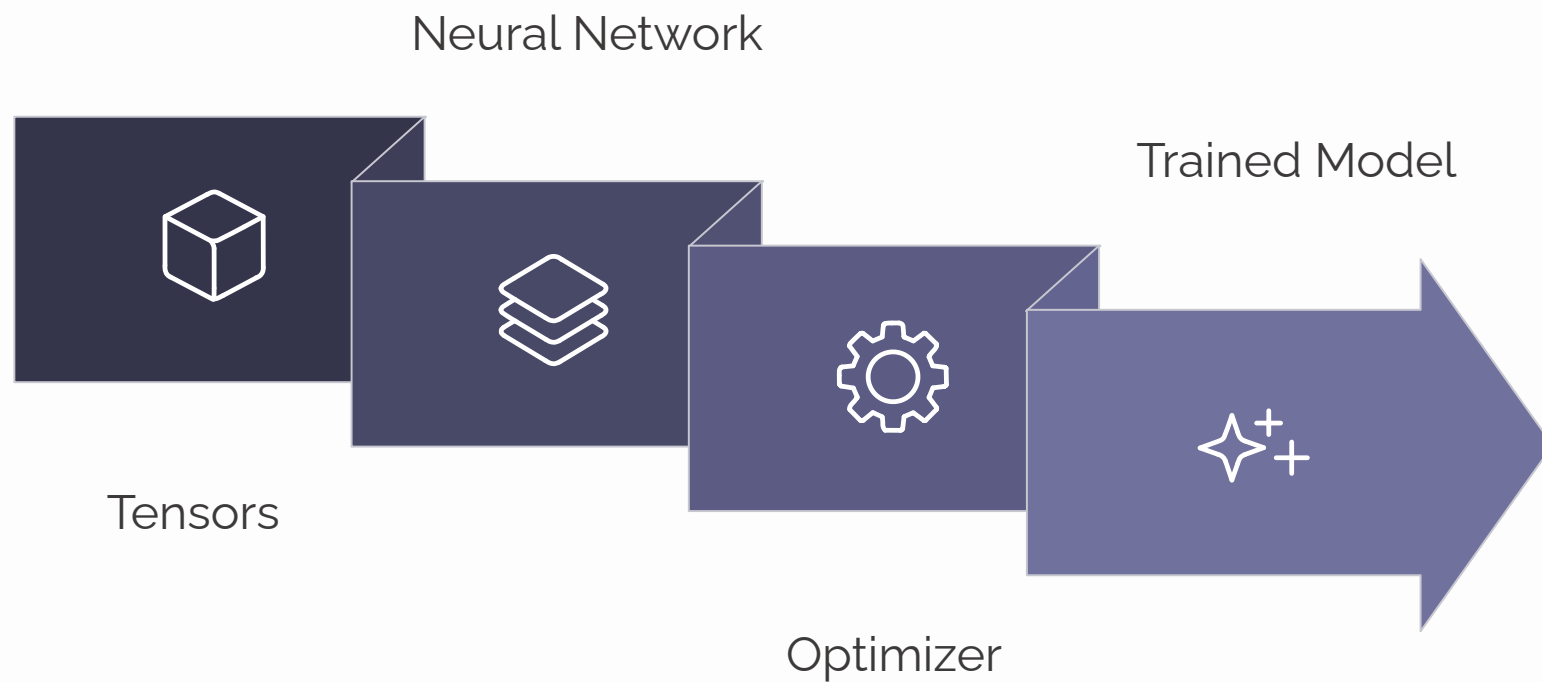
PyTorch handles everything automatically:

```
graph TD; Input --> PyTorch_Neural_Network[PyTorch Neural Network]; PyTorch_Neural_Network --> Loss; Loss --> loss_backward[loss.backward()]; loss_backward --> Optimizer; Optimizer --> Prediction[Prediction ✓];
```

📌 PyTorch does all the heavy math for us — so we focus on building great models!

PyTorch Big Picture & Architecture

Before coding, understand these **3 core building blocks** of PyTorch and how they connect:



Tensors

Store all data as numbers (scalars, vectors, matrices, multi-dimensional arrays)

Autograd

Automatically calculates derivatives/gradients
— no manual math needed

nn.Module

Base class for all neural networks — Linear, CNN, RNN, Transformer, and your custom models

Step 1 & 2 — Install PyTorch & Understand Tensors

Install PyTorch

```
# Install command
pip install torch torchvision torchaudio

# Check version
import torch
print(torch.__version__)
# Output: 2.x.x
```

Check GPU

```
print(torch.cuda.is_available())
# True = GPU ready!
```

What is a Tensor?

A tensor is a **data structure to store numbers** — the foundation of all PyTorch operations.

```
# Scalar (one number)
x = torch.tensor(5)

# Vector (list of numbers)
x = torch.tensor([10, 20, 30])

# Matrix (table of numbers)
x = torch.tensor([[1,2],[3,4]])

# Check shape
print(x.shape)
# torch.Size([2, 2])

# Common tensor creators
torch.zeros(3, 4) # all zeros
torch.ones(2, 3)  # all ones
torch.rand(3, 3)  # random 0-1
```

 Images are stored as tensors! A 28×28 grayscale image = 784 numbers in a tensor.

Step 3, 4 & 5 — GPU, Neural Networks & Activation Functions

Move Tensor to GPU

```
# Select device automatically
device = torch.device(
    "cuda" if torch.cuda.is_available()
    else "cpu"
)
# Move tensor to GPU/CPU
x = torch.tensor([1, 2, 3])
x = x.to(device)
```

Build Your First Neural Network

```
import torch.nn as nn

class MyNetwork(nn.Module):
    def __init__(self):
        super().__init__()
        # Layer 1: 2 inputs → 4 neurons
        self.layer1 = nn.Linear(2, 4)
        # ReLU activation (non-linearity)
        self.relu = nn.ReLU()
        # Layer 2: 4 neurons → 1 output
        self.layer2 = nn.Linear(4, 1)

    def forward(self, x):
        x = self.layer1(x) # Forward
        x = self.relu(x)   # Activate
        x = self.layer2(x) # Output
        return x

model = MyNetwork()
```

Key Concepts

nn.Module — Base class for all neural networks in PyTorch

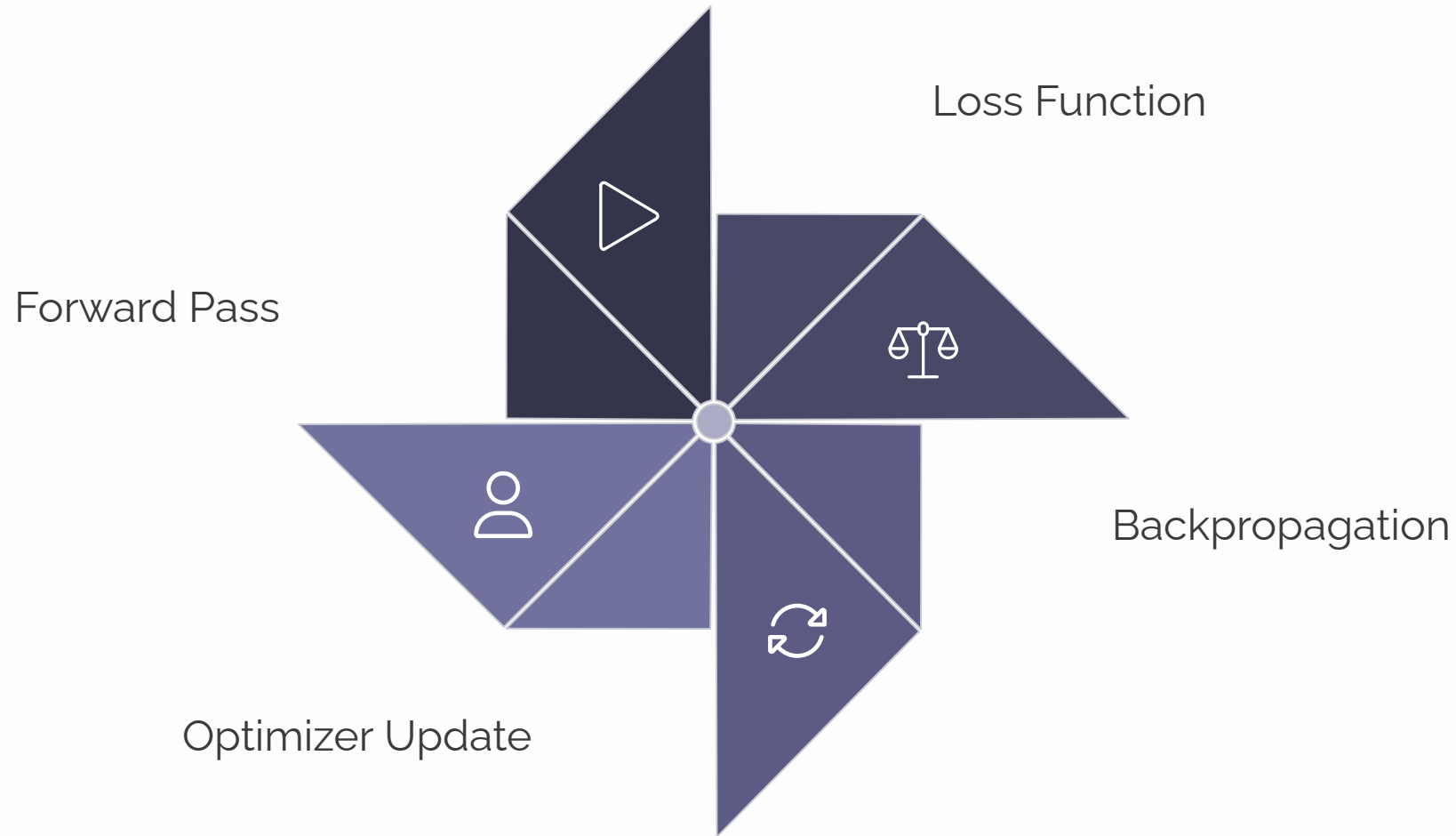
nn.Linear(in, out) — Fully connected layer. Equation: $y = Wx + b$ (Weights $\times$ Input + Bias)

Activation Functions — Add non-linearity so networks learn complex patterns:

- **ReLU**: $\max(0, x)$ — most common hidden layer activation
- **Sigmoid**: binary classification output
- **Softmax**: multi-class classification output

Forward Pass — Input travels through all layers to produce a prediction

Steps 6–9 — Loss, Backpropagation & Optimizer



Loss Function

```
# Regression → MSELoss
loss_fn = nn.MSELoss()

pred = torch.tensor([7.0])
actual = torch.tensor([10.0])
loss = loss_fn(pred, actual)
# Tells us: how wrong is model?
```

Backpropagation

```
# Calculate gradients
loss.backward()
# Gradient = direction to reduce error
# Like finding downhill on a hill
```

Optimizer

```
# Adam optimizer (most popular)
optimizer = torch.optim.Adam(
    model.parameters(),
    lr=0.001 # learning rate
)
# Also: SGD, RMSprop, AdamW
```

The Complete PyTorch Training Loop

This is the **heart of PyTorch**. Remember the **Golden 5 Lines** inside every training loop:

```
import torch
import torch.nn as nn

# Training data: Study Hours → Marks
X = torch.tensor([[1.],[2.],[3.],[4.]])
Y = torch.tensor([[2.],[4.],[6.],[8.]])

# Simple 1-layer model
model = nn.Linear(1, 1)
loss_fn = nn.MSELoss()
optimizer = torch.optim.SGD(
    model.parameters(), lr=0.01
)

# ===== TRAINING LOOP =====
for epoch in range(1000):
    # Step 1: Forward pass → prediction
    prediction = model(X)

    # Step 2: Calculate loss (error)
    loss = loss_fn(prediction, Y)

    # Step 3: Clear old gradients
    optimizer.zero_grad()

    # Step 4: Backpropagation
    loss.backward()

    # Step 5: Update weights
    optimizer.step()

print("Training completed!")

# Test: predict for 5 hours
test = torch.tensor([[5.0]])
result = model(test)
print("Prediction:", result.item())
# Expected output: ≈ 10
```

★ Golden 5 Lines

01

```
prediction = model(X)
```

Forward pass

02

```
loss = loss_fn(pred, Y)
```

Calculate error

03

```
optimizer.zero_grad()
```

Clear old gradients

04

```
loss.backward()
```

Backpropagation

05

```
optimizer.step()
```

Update weights

📋 Forward → Loss → Clear → Backward → Update

Advanced PyTorch — CNN, NLP, Transfer Learning & Deployment

CNN for Images

```
class CNN(nn.Module):
    def __init__(self):
        super().__init__()
        # Convolutional layer
        self.conv1 = nn.Conv2d(1,16,3)
        # Pooling layer
        self.pool = nn.MaxPool2d(2,2)
        # Fully connected output
        self.fc = nn.Linear(16*13*13,10)

    def forward(self, x):
        x = torch.relu(self.conv1(x))
        x = self.pool(x)
        x = torch.flatten(x, 1)
        return self.fc(x)
```

NLP & Transformers

Text → Tokenization → Embeddings →
Transformer → Prediction

PyTorch powers: **BERT, GPT, T5, Vision Transformers, Diffusion Models**

Transfer Learning

```
# Use pretrained model
# Fine-tune on your dataset
# e.g., Plant Disease Detection
```

Save & Load Model

```
# Save after training
torch.save(model.state_dict(),
           "model.pth")
# Load for prediction
model.load_state_dict(
    torch.load("model.pth"))

# Evaluation mode (no gradients)
model.eval()
with torch.no_grad():
    prediction = model(X)
```

DataLoader

```
from torch.utils.data import DataLoader

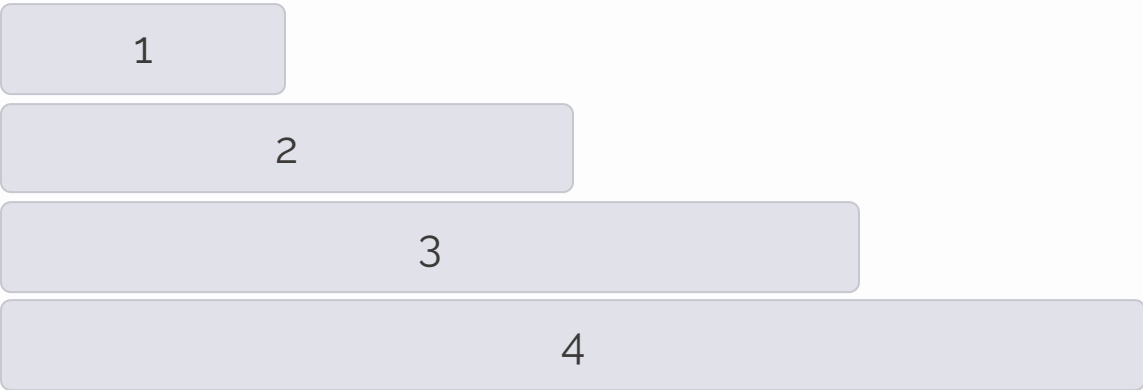
# Load data in mini-batches
loader = DataLoader(
    dataset,
    batch_size=32, # 32 images/batch
    shuffle=True # random order
)
# Common batch sizes: 16,32,64,128
```

Industry Applications

- Face recognition & medical imaging
- Autonomous vehicles
- Chatbots & LLMs
- Industrial defect detection
- Robotics & navigation

PyTorch Learning Roadmap & Key Terms

🎓 B.Tech Practical Sequence (18 Levels)



- 1 Levels 1–4
Python Basics → Tensors → Tensor Operations → Autograd
- 2 Levels 5–10
nn.Module → Linear Layers → Activations → Loss → Optimizers → Training Loop
- 3 Levels 11–14
Dataset + DataLoader → CNN → RNN/LSTM → Transfer Learning
- 4 Levels 15–18
Transformers → GPU/CUDA → Fine-Tuning → Deployment

★ One Sentence to Remember PyTorch

"PyTorch takes data as tensors, sends it through a neural network, calculates the error, uses backpropagation to calculate gradients, and uses an optimizer to update the weights until the model learns."

📋 Key Terms Quick Reference

Term	Simple Meaning
Tensor	Stores numerical data
nn.Module	Base class for neural networks
Forward Pass	Making a prediction
Loss	Measures prediction error
Autograd	Auto-calculates gradients
Optimizer	Updates weights
Epoch	One full training cycle
Transfer Learning	Reusing a pretrained model